第 11 天 • 11/12 • 今日知识点回顾

筛选与排序：布尔索引、query、loc/iloc、多列排序

目标：用三种写法（布尔索引 / query / loc）写出同一个筛选条件，并说清各自的适用场景。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 用三种写法（布尔索引 / query / loc）写出同一个筛选条件，并说清各自的适用场景。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：3 集 • 合计约 37 分钟（边看边在本机敲一遍）

P23	Pandas2数据选择操作	约 18 分钟
P24	Pandas2复杂数据查询操作	约 15 分钟
P28	Pandas2高级查询过滤	约 4 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch3.rst`

第三章 索引：loc / iloc / 布尔索引 / 多层索引

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 布尔索引：一行写完所有条件

- 一句话：df[df['列'] > 值] 是核心写法；多个条件用 & | ~，每个条件都要加括号。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
print(len(df[df['amount'] > 2000]))
print(len(df[(df['status'] == 'paid') & (df['city'] == '北京')]))
print(len(df[(df['status'] == 'paid') | (df['amount'] > 5000)]))
print(len(df[~(df['status'] == 'refunded')]))
```

知识点 1/7 • 布尔索引：一行写完所有条件（续）

★ 不加括号会报错：

• ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(),a.item(), a.any() or a.all(). 原因是 & 的优先级比 > 高，Python 先算 'paid' & df['city']，整串就乱了。

知识点 2/7 • query：条件写成字符串，适合「条件很多、还要给业务看」

- 一句话：df.query("amount > 2000 and city == '北京'")，用 and/or 不用 &/|。
- 代码：

```
print(len(df.query("amount > 2000")))
print(len(df.query("status == 'paid' and city == '北京'")))
city = '上海'
print(len(df.query("city == @city")))      # 用 @ 引用外部变量
```

★ query 里列名有空格或中文要反引号：df.query("`销售员` == '孙悦'")。

- 本机 clean_v2.csv 的列名是英文，暂时用不上，但读 Excel 时天天遇到。

知识点 3/7 • loc 与 iloc：一个按名字、一个按位置

- 一句话：loc[行标签, 列名]；iloc[行号, 列号] —— 行号从 0 开始，不含右端。
- 代码：

```
d2 = df.set_index('order_id')
print(d2.loc[100001, ['city', 'amount']].to_dict())
print(d2.iloc[0:2, 0:3].to_string())
print(df.loc[0:2, 'city'].tolist())        # loc 的切片**含右端**
```

★ 最容易错的一条：loc[0:2] 取 3 行（含 2），iloc[0:2] 取 2 行（不含 2）。

- 记法：loc 是「标签切片」，像字典；iloc 是「位置切片」，像 list。

知识点 4/7 • 排序：sort_values 多列 + 稳定排序

- 一句话：df.sort_values(['列1', '列2'], ascending=[True, False])。
- 代码：

```
top = (df.sort_values(['amount', 'order_id'], ascending=[False, True])
.head(5)[['order_id', 'city', 'amount']])
print(top.to_string(index=False))
print(df.nlargest(3, 'amount')[['order_id', 'amount']].to_string(index=False))
```

★ nlargest/nsmallest 比 sort_values().head() 快（内部用堆，不全排），只要前 N 名时优先用它。sort_values 默认 ascending=True，写反了会得到最小的一批。

知识点 5/7 • isin / between / str.contains：三个高频条件写法

- 一句话：isin 判「属于集合」，between 判区间（含两端），str.contains 做模糊匹配。
- 代码：

```
print(len(df[df['city'].isin(['北京', '上海', '广州'])]))
print(len(df[df['amount'].between(1000, 2000)]))
print(len(df[df['product_name'].str.contains('小米', na=False)]))
print(len(df[df['product_name'].str.startswith('罗技', na=False)]))
```

★ str.contains 默认 na=nan 会返回 nan，在布尔索引里被当成 False —— 看着没事，但 df[mask] 的行数会比你算的少。一律显式写 na=False。

知识点 6/7 • 真数据：几个能直接拿去讲的筛选结果

- 一句话：每个筛选都要能说出「筛出来多少行、为什么是这个数」。
- 代码：

```
paid = df[df['status'] == 'paid']
print('paid 行数', len(paid), '合计', round(paid['amount'].sum(), 2))
print('北京 paid', len(paid[paid['city'] == '北京']))
print('金额 1000-2000', len(df[df['amount'].between(1000, 2000)]))
print('品类是数码', len(df[df['category'] == '数码']))
```

- （本机实测 clean_v2.csv：paid 417 行，合计 422,436.00；
- 北京 paid 若干行；金额 1000-2000 的行数可以直接数。
- 注意 paid 合计要和第 12 天 groupby 的结果一致 —— 不一致就是筛选口径不同。）

知识点 7/7 • 筛选完记得 copy，别改到原表

- 一句话：df[mask] 拿到的是视图或副本不确定，要改就显式 .copy()。
- 代码：

```
sub = df[df['status'] == 'paid'].copy()
```

知识点 7/7 • 筛选完记得 copy，别改到原表（续）

- sub['amount_disc'] = sub['amount'] * 0.9　　# 安全

```
# 不 copy 直接赋值会报 SettingWithCopyWarning
```

★ pandas 3.0 强制开启 Copy-on-Write，SettingWithCopyWarning 基本消失，但「改了子集、原表没变」这个语义更容易让人困惑 —— 显式 .copy() 最稳。

代码速查：今天这几段，要在本机跑通（1/5）

写 filter_practice.py，每题打印行数 + 一行解释：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

1) 三种写法写同一个条件：status=paid 且 city=北京

```
a = len(df[(df['status'] == 'paid') & (df['city'] == '北京')])
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
b = len(df.query("status == 'paid' and city == '北京'"))
```

```
c = len(df.loc[(df['status'] == 'paid') & (df['city'] == '北京')])
```

```
print('三种写法：', a, b, c)          # 三个数必须相同
```

2) 金额 Top 10，用 nlargest

```
print(df.nlargest(10, 'amount')[['order_id', 'city', 'category', 'amount']].to_string(index=False))
```

3) 区间 + 集合 + 模糊

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
print('1000-2000: ', len(df[df['amount'].between(1000, 2000)]))
```

```
print('北上广: ', len(df[df['city'].isin(['北京', '上海', '广州'])]))
```

```
print('商品名含 小米: ', len(df[df['product_name'].str.contains('小米', na=False)]))
```

4) 排除退款

```
print('非退款行数: ', len(df[df['status'] != 'refunded']))
```

5) 按两列排序取前 5

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
print(df.sort_values(['city', 'amount'], ascending=[True, False])
```

```
        .groupby('city').head(1)[['city', 'order_id', 'amount']]
```

.to_string(index=False))

验收（做到才算过关）：

① 第 1 题三个数完全一致；

```
② 能说清为什么 (df['a'] > 1) & (df['b'] < 2) 不加括号会报 ValueError；
```

代码速查：今天这几段，要在本机跑通（5/5）

```
③ 第 5 题能解释 groupby('city').head(1) 和直接 head(1) 的差别。
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 最容易错的一条：loc[0:2] 取 3 行（含 2），iloc[0:2] 取 2 行（不含 2）。

注意 paid 合计要和第 12 天 groupby 的结果一致 —— 不一致就是筛选口径不同。）

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料说出：注意 paid 合计要和第 12 天 groupby 的结果一致 —— 不一致就是筛选口径不同。）
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 12 天 • groupby 分组聚合：agg、transform、多列分组

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-12-第11天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data